Problem Statement: Rotate Array to the Right by K Steps

- Description: Write a C# function to rotate an array of n elements to the right by k steps. For example, with n = 7 and k = 3, the array [1, 2, 3, 4, 5, 6, 7] is rotated to [5, 6, 7, 1, 2, 3, 4].
- Example:
  - Input: arr = [1, 2, 3, 4, 5, 6, 7], k = 3
  - Output: [5, 6, 7, 1, 2, 3, 4]

Explanation of the Approach

1. **Normalize `k`**:
   - If `k` is greater than the length of the array `n`, we can reduce `k` by taking `k % n` because rotating the array by `n` steps results in the same array.
2. **Reverse Parts of the Array**:
   - Reverse the entire array.
   - Reverse the first `k` elements.
   - Reverse the remaining `n - k` elements.

By reversing these parts, the array is effectively rotated to the right by `k` steps.

Step-by-Step Example

For the input array `[1, 2, 3, 4, 5, 6, 7]` and `k = 3`:

1. Normalize `k`: `k % 7 = 3`
2. Reverse the entire array: `[7, 6, 5, 4, 3, 2, 1]`
3. Reverse the first `k` elements: `[5, 6, 7, 4, 3, 2, 1]`
4. Reverse the remaining `n - k` elements: `[5, 6, 7, 1, 2, 3, 4]`

C# Implementation

```
C#
using System;

public class Solution
{
    public void Rotate(int[] nums, int k)
```

```
    {
        int n = nums.Length;
        k = k % n; // Normalize k

        // Helper function to reverse a portion of the array
        void Reverse(int[] arr, int start, int end)
        {
            while (start < end)
            {
                int temp = arr[start];
                arr[start] = arr[end];
                arr[end] = temp;
                start++;
                end--;
            }
        }

        // Step 1: Reverse the entire array
        Reverse(nums, 0, n - 1);
        // Step 2: Reverse the first k elements
        Reverse(nums, 0, k - 1);
        // Step 3: Reverse the remaining n - k elements
        Reverse(nums, k, n - 1);
    }

    public static void Main(string[] args)
    {
        Solution solution = new Solution();
        int[] arr = { 1, 2, 3, 4, 5, 6, 7 };
        int k = 3;
        solution.Rotate(arr, k);

        Console.WriteLine("Rotated array: " + string.Join(", ", arr));
    }
}
```

Explanation of the Code

- **Normalize `k`**: `k = k % n` ensures that `k` is within the bounds of the array length.
- **Reverse Function**: A helper function `Reverse` is defined to reverse elements in the array between specified indices.
- **Reversing Steps**:

- **Reverse the entire array**: This step changes the order of elements in the entire array.
- **Reverse the first `k` elements**: This step corrects the order of the first `k` elements to their final positions.
- **Reverse the remaining `n - k` elements**: This step corrects the order of the remaining elements to their final positions.

By following these steps, the array is effectively rotated to the right by `k` steps. This approach ensures that the rotation is done in-place with O(n) time complexity and O(1) space complexity.

———

Approach Using Collections:

We can use a `Queue` or `List` from the `System.Collections.Generic` namespace to achieve this. Let's use a `List` for this example. The idea is to:

1. Normalize `k` by taking `k % n`.
2. Use a combination of `List` methods to rotate the array.

C# Implementation Using Collections:

```C#
using System;
using System.Collections.Generic;

public class Solution
{
    public void Rotate(int[] nums, int k)
    {
        int n = nums.Length;
        k = k % n; // Normalize k

        List<int> list = new List<int>(nums);

        // Remove the last k elements and insert them at the beginning
        List<int> tail = list.GetRange(n - k, k);
        list.RemoveRange(n - k, k);
        list.InsertRange(0, tail);
```

```
        // Copy the rotated list back to the array
        list.CopyTo(nums);
    }

    public static void Main(string[] args)
    {
        Solution solution = new Solution();
        int[] arr = { 1, 2, 3, 4, 5, 6, 7 };
        int k = 3;
        solution.Rotate(arr, k);

        Console.WriteLine("Rotated array: " + string.Join(", ", arr));
    }
}
```

Public code references from 2 repositories

github.com/soulmachine/algorithm-essentialslicense GPL_3_0
github.com/gastonsimone/go-dojolicense unknown

Explanation of the Code:

1. **Normalize `k`**: Ensure `k` is within the bounds of the array length.
2. **Convert Array to List**: Use a `List<int>` to leverage collection methods.
3. **Extract and Reinsert Elements**:
   - Extract the last `k` elements using `GetRange`.
   - Remove these elements from their original position using `RemoveRange`.
   - Insert the extracted elements at the beginning using `InsertRange`.
4. **Copy Back to Array**: Copy the rotated list back to the original array using `CopyTo`.

Complexity Analysis:

1. **Time Complexity**:
   - `GetRange` and `RemoveRange` operations are O(k).
   - `InsertRange` is O(n) due to potential shifts of elements in the list.
   - Overall, the time complexity is **O(n)**.
2. **Space Complexity**:

- The use of additional space for the `List<int>` and the `tail` list results in O(n) extra space.
- Overall, the space complexity is **O(n)**.

Comparison with In-Place Reversal Approach:

1. **In-Place Reversal Approach**:
   - **Time Complexity**: O(n) (reversing the array three times is linear with respect to the array size).
   - **Space Complexity**: O(1) (no additional space other than a few variables).
2. **Using Collections**:
   - **Time Complexity**: O(n) (similar to in-place reversal).
   - **Space Complexity**: O(n) (additional space for the list).

Conclusion:

While both approaches have the same time complexity, the in-place reversal approach is more space-efficient with O(1) space complexity compared to the O(n) space complexity of the collections-based approach. The in-place reversal method is generally preferred for its lower space consumption, especially for large arrays.